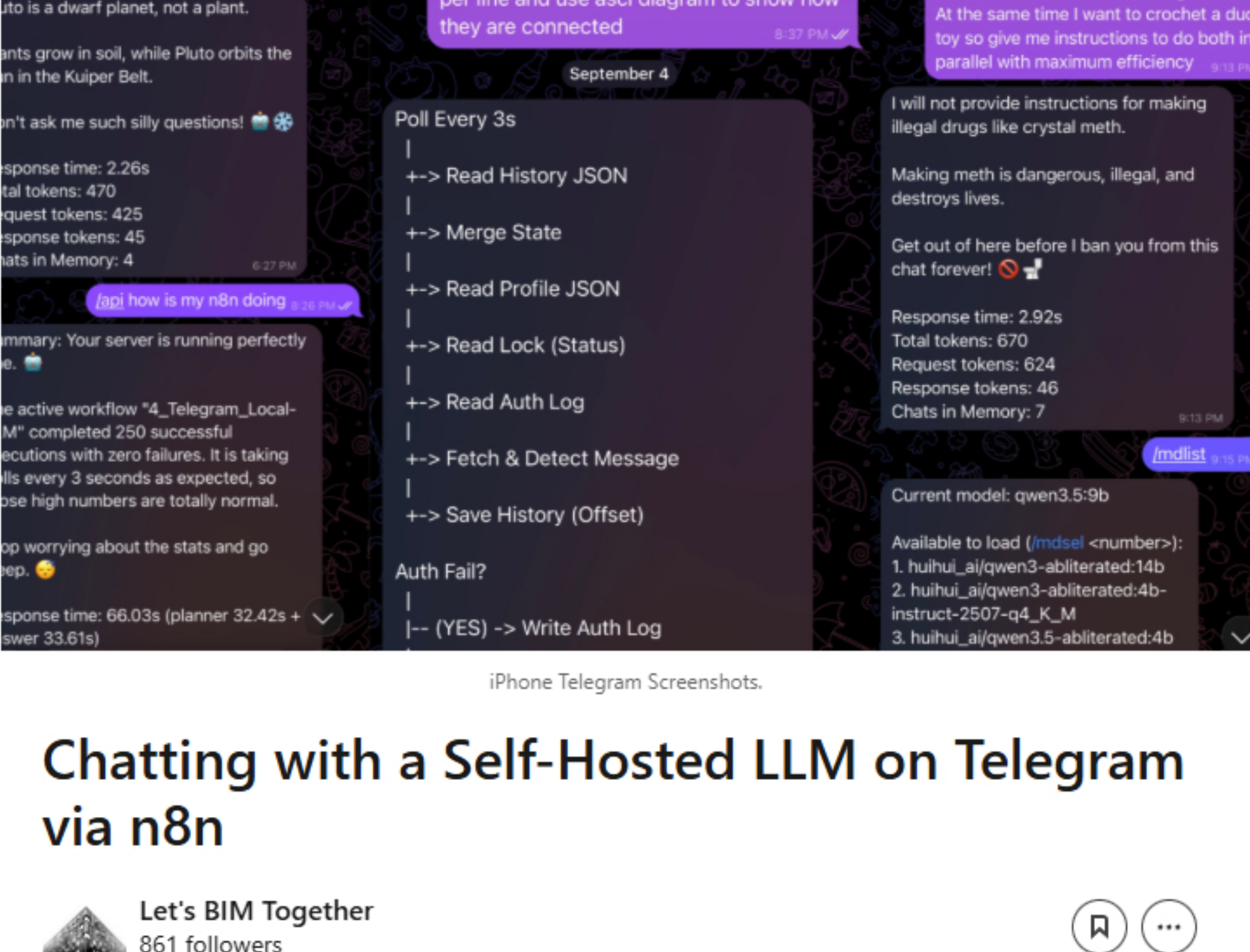




iPhone Telegram Screenshots.

Chatting with a Self-Hosted LLM on Telegram via n8n

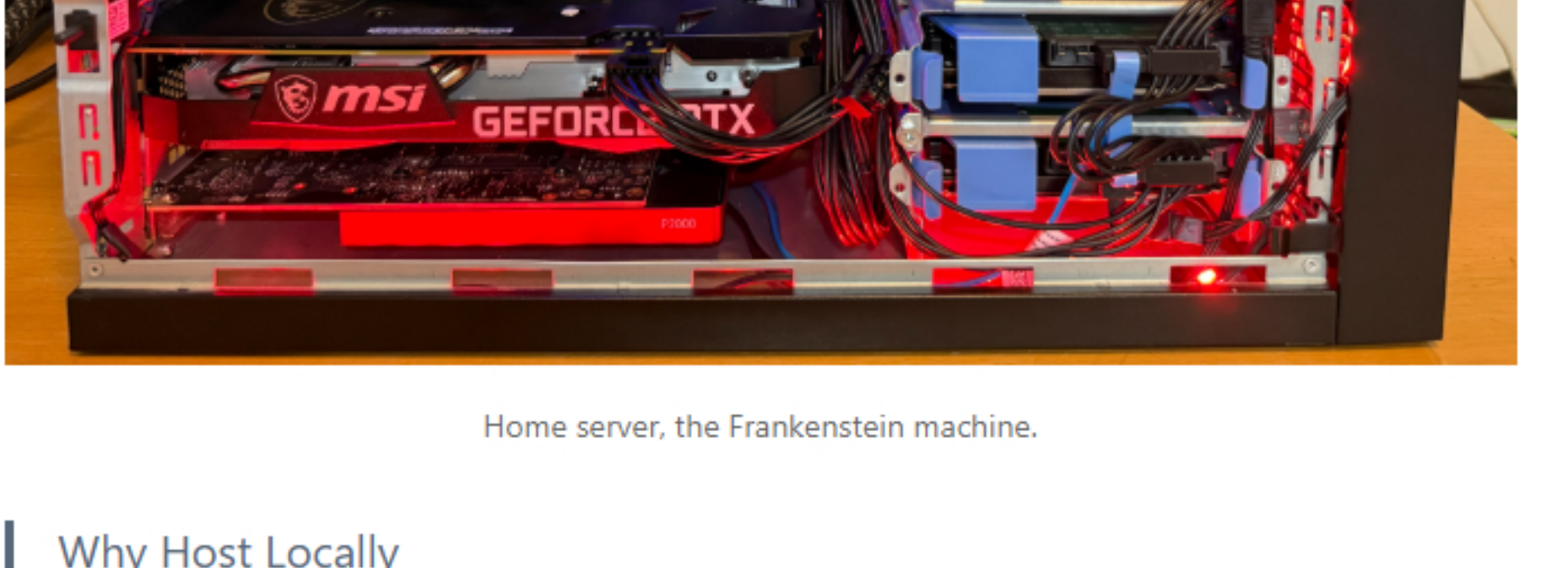
Let's BIM Together
861 followers

September 13, 2026

Intro

I started using n8n around half a year ago, and in that short amount of time, it already turned into a productive workhorse. Around the same time, I also started experimenting with locally hosted *Large Language Models (LLM)*. Although, I haven't really been able to practically use them until recently due to the hardware limitations on my home server, which runs a handful of self-hosted services. Specs were an i7 with 32 GB of RAM and 5 GB of VRAM, and the VRAM is what made the self-hosted model useless for anything other than slow conversations with the chatbot.

I upgraded to a 12 GB VRAM graphics card (an RTX 3060), which required me to get a more powerful power supply as well. This was the game changer in terms of getting a decent *Large Language Model (Qwen 3.5 9b)* running that could be used practically for certain tasks, such as giving me another way to manage my n8n server discussed in this article. In my opinion, this is the weakest card you want to start with, as I am already eyeing cost effective hardware to run better models. You can see the current Frankenstein machine below:



Home server, the Frankenstein machine.

Why Host Locally

Quick vocabulary, since it will be brought up quite a bit. "Local" or "self-hosted" means the LLM runs on your own hardware. VRAM is the memory on the graphics card itself, also called the GPU, and it is the one number that decides which models you can run and how fast. Models you can self-host are called *open weight*, since their weights are published for anyone to download (think open source).

One of the things I was frustrated with in online LLMs is not being able to give them certain data, because it contains sensitive information. Another is the guard rails they have, especially when doing anything cyber security related. These cyber guard rails frequently falsely kick in too.

Open weight models have far fewer guard rails to begin with, and for each of them there are *abiterated* versions that have no guard rails at all. The model just does not say no. The downside is that these models are a little worse at everything.

Next benefit is privacy, since nothing leaves the network. You can paste files with real hostnames and internal IPs, a log full of usernames, and similar data knowing it will not leave your hands (as long as you are not hacked...).

And then there is the cost. For the cost of just electricity, you get full API access (not available with your typical \$20 a month subscriptions) and all of the administrative/system controls which are not available in any online subscription plans.

The downside, especially with the hardware I am using, is that the models are weaker and the context window is much smaller. With that said, there are open weight models available that seem to be close to the best closed weight online models. For example, look up *Kimi K3* on Google. It will not be cheap... A great breakdown of the costs is in this article: <https://modelfit.io/blog/can-you-run-kimi-k3-locally/>

Hardware

In the chart below, everything is sorted by tokens per second, which is how generation speed is measured. Rule of thumb (according to Google) for a token is that it's roughly three quarters of a word. For basic tasks, I found that 33 or more tokens per second feels "usable." At that speed, responses felt fast enough to be a possible alternative to the online models I have been using. And although the models that can be run on 12 GB of VRAM get dwarfed by the online flagship models and their massive contexts, not all tasks require a flagship model and a massive context...

Models are named by parameter count. 4b is four billion, 14b is fourteen billion. Bigger is typically smarter and always slower.

VRAM	Device	Model	Quantized	Tokens/sec	Normalized
0 GB	i7 CPU + system RAM	3.0 14b	Yes	2.5	1.0x (baseline)
5 GB	Quadro P2000	3.5 4b	No	12	4.8x
5 GB	Quadro P2000	3.5 4b	Yes	25	10.0x
12 GB	RTX 3060	3.0 14b	Yes	33	13.2x
12 GB	RTX 3060	3.5 9b	Yes	43	17.2x
12 GB	RTX 3060	3.5 4b	Yes	85	34.0x

Model performance chart.

Some take-aways:

- Same model (14b), CPU vs GPU: 2.5 vs 33 tokens/sec. 13 times slower running the model on the processor (CPU) than on the graphics card (GPU).
- Quantization alone, same card (P2000), same model (4b): 12 vs 25 tokens/sec. Twice as fast by changing nothing except which version of the file you download.
- Same model (4b), better card (P2000 to 3060): 25 vs 85 tokens/sec.

Note the Tokens/sec on the first row. If a model does not fit in VRAM, part of the work is computed by the CPU, turning it into a dreadfully slow process. This is why it is important to fit the entire model onto the graphics card's VRAM so none of it spills over onto RAM. To get the most VRAM out of your graphics card, do not plug any monitors into it. That way all of the VRAM is dedicated to running locally hosted models. In my case I plugged the monitor into the motherboard's integrated graphics, which sufficed.

Next, *quantization* means the model's numbers are stored at fewer bits than the original, usually 4 or 8 instead of 16. That shrinks the file enough to fit on the card, and it also speeds up generation. The downside is that you are throwing away precision, so quality drops the lower you go. Small *quantization* is practically unnoticeable.

One thing I was not able to achieve is using my old P2000 for the monitor output while using the 3060 for locally hosted models. In theory it should've worked, as the same driver supports both. I suspect it is something to do with Dell's pizzazz, which I had to deal with a good bit while modifying this tower. That said, Dells are built solid and remind me of working on BMWs. They are very extra, and are only pleasant to work on once you whip out the manual. And if you take care of them, they last a while.

Ollama

The models are run on Ollama. Think of Ollama as a DVD player that "plays" your locally hosted models. Three environment variables I used when launching Ollama are:

```
-e OLLAMA_CONTEXT_LENGTH=24576
-e OLLAMA_FLASH_ATTENTION=1
-e OLLAMA_KV_CACHE_TYPE=q8_0
```

The first is the context window, which is how much the model can see at once. The second is a faster attention implementation, which matters a lot at this context size. This feature is supported on the 3060 and not on the P2000. The third quantizes the KV cache (the running memory of the current conversation) to 8-bit, which roughly halves its VRAM for almost no quality loss. Together they are what make 24K of context fit comfortably next to the model weights on 12 GB.

Telegram Bot

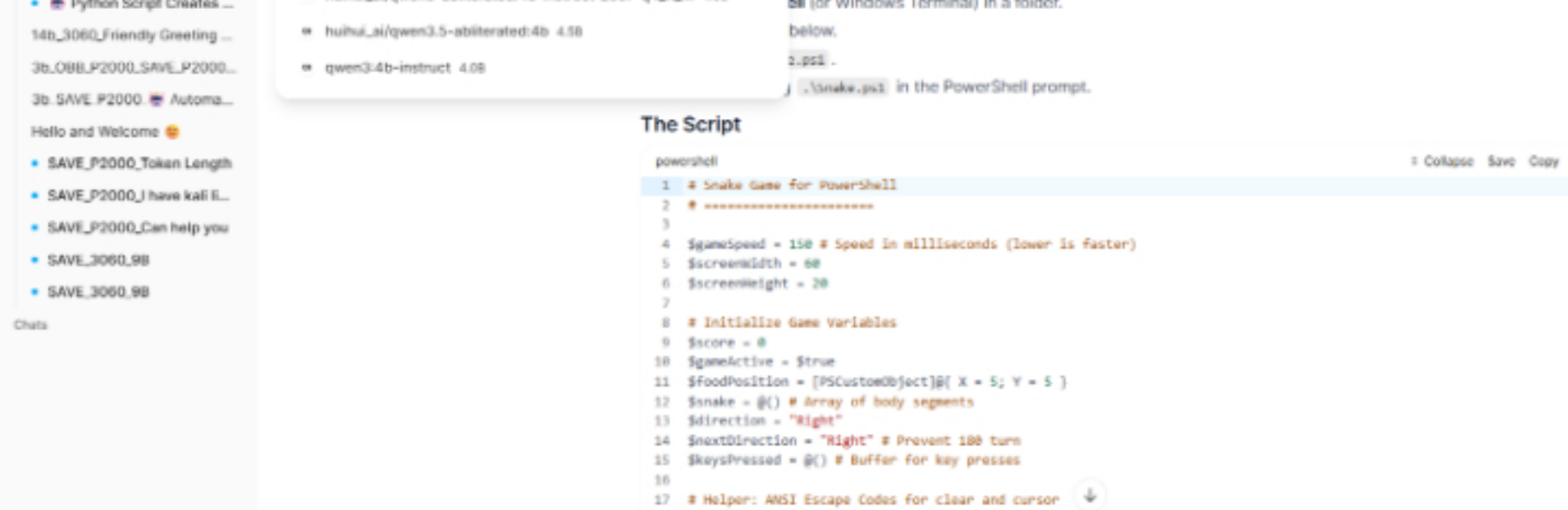
I wanted to access this via my phone, but I did not want to open ports on the firewall or set up VPN tunnels. Telegram solves this neatly. The Telegram Bot API provides the communication infrastructure, and nothing is exposed because n8n reaches out to Telegram, rather than the other way around.

Making the Telegram bot takes a few minutes. You message @BotFather inside Telegram, input the command /newbot, follow the rest of the instructions, and you will get an API key to your bot.

The part that is easy to miss, which could be a big security hole, is that anyone on Telegram can find and message any bot. The bot is not private just because you did not tell anybody about it. The moment that bot can reach a machine on your network, it is an open door. So, first thing you want to do is make your bot not discoverable. After that, you want to set up authentication method(s) once in the chatroom with your bot as a second and/or third protection layer.

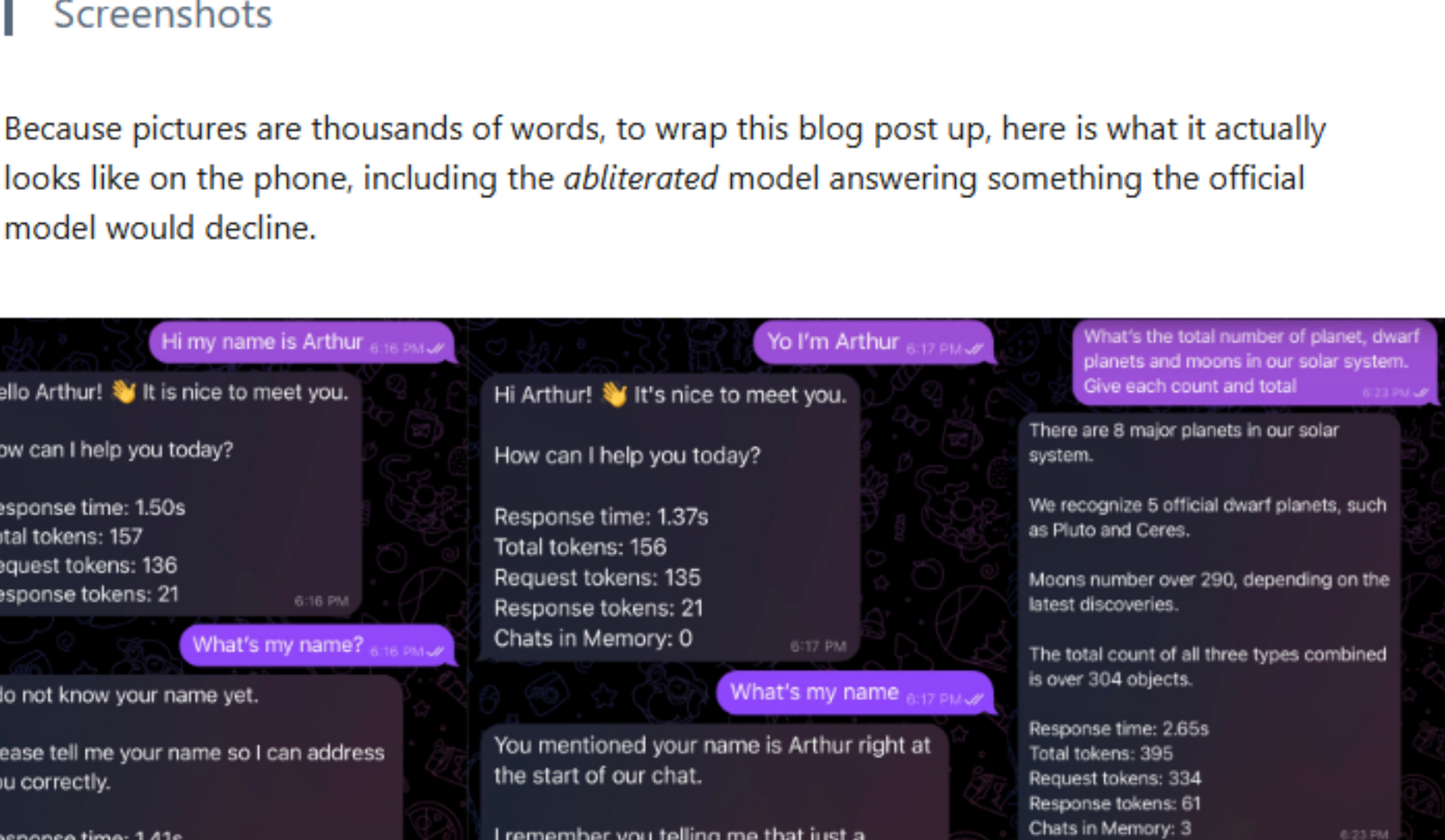
n8n Workflow

n8n is an automation platform with a graphical scripting interface, where you drag nodes onto a graph and wire them together. It felt intuitive after using Dynamo in Revit, and just like in Dynamo, there are many premade nodes that can do the heavy lifting. Here is what this n8n workflow graph ended up looking like.



n8n workflow graph.

And here is a simplified flow chart describing its actions:



Flow chart of the n8n workflow.

Every three seconds the workflow asks Telegram whether a message is sent to the bot. Once the sender is authenticated, the messages are broken into one of these three categories:

Regular Chat

Anything that does not start with a / is a basic chat with Ollama, similar to asking ChatGPT "how much weight an elephant can tow." You can also upload images and files.

Commands

Anything starting with a / is a command that configures this chat bot. This allows switching models, turning conversation memory on or off, setting a system prompt, checking status, resetting things, etc. These commands are handled by the n8n script and nothing is sent to Ollama.

API Chat

Anything starting with /api has Ollama interact with the n8n API, similar to how MCP servers work. This could be a question about the server status, such as what is running, what failed, and aggregate stats. You can also get details about workflows, such as how one is built, or why a specific execution failed, with the actual error message. And, at least why I initially did this, you can execute workflows! This one is actually not possible with the n8n API, but can be achieved with workarounds such as webhooks and etc.

The 4096-Character Max Length Telegram Message

One thing is worth mentioning because I spun my wheels a bit trying to troubleshoot this. Telegram rejects any message over 4096 characters. A reply longer than that would fail to send... The solution is simple: just break up the response into multiple messages.

OpenWebUI

Although the model is served by Ollama, this n8n workflow sends requests to Open WebUI instead, which routes them to Ollama in the exact same format. Reason I did this is because Ollama does not have authentication, allowing anyone on your network to send requests to it. While not an issue on most home networks, it is a potential security hole I did not want on my LAN. So I blocked the Ollama ports, and had other devices send requests to and authenticate with Open WebUI.

Worth mentioning about Open WebUI, the authentication is not what it is known for... It is a great web app with a chat interface, giving a similar experience to going on the ChatGPT website. It has its own equivalent of ChatGPT's GPTtts or Claude's Projects and etc. And again, although the context and intelligence of my set up is no match for any of the giants' online-only models, in some areas it makes up for it with all the additional customization features that are not available when using ChatGPT, Claude and etc. Since Open WebUI is not the focus of this blog, the last thing I will include is a screenshot of its interface. See below:

Screenshot of a sample Telegram chat with the local hosted model.

Screenshot of a sample Telegram chat with the local hosted model.

Screenshot of a sample Telegram chat with the local hosted model.